

Universidad Técnica Estatal de Quevedo
FORM-IC-01 — Investigación Individual
Arquitecturas Distribuidas en la Industria
Análisis de la Arquitectura de Microservicios en Netflix

Jhinson Stalyn Aucatoma Celorio
7mo Software “A”

Martes 2 de Junio del 2026

[Ver repositorio en GitHub](#)

Índice

1. Introducción	2
2. Descripción del caso	2
3. ¿Por qué eligieron la arquitectura de microservicios?	2
3.1. Escalabilidad independiente	3
3.2. Equipos de desarrollo autónomos	3
3.3. Despliegues continuos e independientes	3
3.4. Mayor resiliencia y tolerancia a fallos	3
3.5. Flexibilidad tecnológica	3
4. Beneficios obtenidos	3
5. Desafíos encontrados	4
6. Aplicación al PFC	5

1. Introducción

La arquitectura de microservicios se ha convertido en uno de los enfoques más utilizados para construir sistemas distribuidos escalables y resilientes. Empresas como Netflix han adoptado este estilo arquitectónico para gestionar millones de usuarios y garantizar alta disponibilidad. Según Henríquez et al. [1], la selección de un estilo arquitectónico adecuado es fundamental para garantizar la escalabilidad, flexibilidad, mantenibilidad y eficiencia de un sistema de software. La arquitectura de microservicios, como evolución de la arquitectura orientada a servicios (SOA), descompone las aplicaciones en componentes pequeños e independientes que pueden desplegarse de forma autónoma y escalar horizontalmente. El presente trabajo analiza la implementación de microservicios en Netflix, los motivos de su adopción, los beneficios obtenidos, los desafíos enfrentados y su posible aplicación al proyecto integrador TiendaTech.

2. Descripción del caso

Netflix es una de las principales plataformas de streaming de contenido en video a nivel mundial, con presencia en más de 190 países. Su infraestructura tecnológica ha evolucionado de forma significativa a lo largo de los años para responder al crecimiento exponencial de su base de usuarios y al aumento en la demanda de contenido. Katkuri [2] señala que la transformación de Netflix ha habilitado el procesamiento de más de 450 petabytes de datos por día a través de su infraestructura de streaming, lo que evidencia la escala masiva de sus operaciones.

Originalmente, Netflix operaba bajo una arquitectura monolítica donde todos los servicios estaban fuertemente acoplados en una única aplicación, lo que dificultaba la escalabilidad y la tolerancia a fallos. A partir de 2010, la empresa inició su migración hacia Amazon Web Services (AWS), separando progresivamente las funciones críticas del sistema. Entre 2011 y 2012 se introdujeron los primeros microservicios independientes, cada uno gestionado por equipos autónomos. Esta transformación continuó con la incorporación de herramientas de automatización y tolerancia a fallos entre 2013 y 2015, y se refinó con soluciones de orquestación como Netflix Conductor a partir de 2016 [1].

En la actualidad, Netflix opera con cientos de microservicios independientes que trabajan de forma coordinada para brindar una experiencia de streaming fluida. La plataforma emplea herramientas como Eureka para el descubrimiento de servicios, Kafka para el procesamiento de eventos, Spinnaker para el despliegue continuo, Chaos Monkey para las pruebas de resiliencia, y un API Gateway para la gestión del tráfico. [2].

3. ¿Por qué eligieron la arquitectura de microservicios?

La decisión de Netflix de adoptar una arquitectura de microservicios surgió como respuesta a los desafíos que enfrentaba su sistema monolítico a medida que la plataforma crecía y aumentaba la demanda de usuarios. A continuación, se presentan las principales razones que impulsaron este cambio de arquitectura.

3.1. Escalabilidad independiente

El sistema monolítico original de Netflix presentaba degradación significativa del rendimiento cuando la carga de usuarios superaba las 3,000 solicitudes concurrentes, con tiempos de respuesta que aumentaban un 32 % por cada 1,000 usuarios adicionales [2]. Los microservicios permiten escalar cada componente de forma autónoma según la demanda, sin necesidad de replicar toda la aplicación.

3.2. Equipos de desarrollo autónomos

Bajo el liderazgo del arquitecto de nube Adrian Cockcroft, Netflix adoptó un modelo en el que equipos pequeños e independientes gestionaban servicios específicos, lo que aceleró el desarrollo y la innovación de nuevas funcionalidades. Una práctica clave fue la asignación de almacenes de datos dedicados para cada microservicio, eliminando dependencias y otorgando a los equipos la flexibilidad de elegir la base de datos más adecuada para sus necesidades [1].

3.3. Despliegues continuos e independientes

En la arquitectura monolítica, los despliegues mayores requerían ventanas de tiempo de hasta 6.5 horas, con una tasa de reversión del 28 % debido a problemas de integración [2]. Con microservicios, Netflix pasó de realizar pocas liberaciones semanales a más de 1,000 despliegues diarios, manteniendo una tasa de éxito del 99.6 % [2]. Los microservicios facilitan la integración y el despliegue continuo (CI/CD), permitiendo actualizaciones independientes de cada componente.

3.4. Mayor resiliencia y tolerancia a fallos

La arquitectura monolítica no permitía aislar fallos: un error en un módulo podía comprometer toda la aplicación. Con microservicios y patrones como los *circuit breakers*, Netflix logró reducir los fallos a nivel de sistema en un 78 %, y el tiempo promedio de resolución de incidentes bajó de 55 minutos a tan solo 12 minutos [2]. Herramientas como Chaos Monkey permitieron realizar pruebas proactivas de resiliencia, con un promedio de 5,000 experimentos automatizados mensuales, lo que contribuyó a una reducción del 71 % en incidentes de producción [2].

3.5. Flexibilidad tecnológica

La arquitectura de microservicios permite que cada servicio utilice la tecnología y la base de datos más apropiada para su función específica, optimizando el rendimiento y la gestión de datos. Esto habilitó a Netflix a incorporar nuevas capacidades como el procesamiento serverless para cargas de trabajo de machine learning y analítica, sin necesidad de modificar el resto del sistema [1].

4. Beneficios obtenidos

La transición a microservicios generó mejoras concretas y medibles en múltiples dimensiones operativas de Netflix. La Tabla 1 resume los principales beneficios documentados en los estudios analizados.

Cuadro 1: Principales beneficios obtenidos con la arquitectura de microservicios en Netflix

Beneficio	Descripción	Métrica documentada
Escalabilidad	Cada servicio escala de forma independiente según la demanda, sin afectar al resto del sistema.	Manejo de picos de hasta 925,000 transmisiones simultáneas [2].
Alta disponibilidad	Los fallos quedan aislados en servicios individuales gracias a los circuit breakers y la tolerancia a fallos.	Tasa de disponibilidad del 99.99 % frente al sistema monolítico [2].
Flexibilidad tecnológica	Cada microservicio puede utilizar la tecnología y base de datos más adecuada para su función.	Reducción del 43 % en tiempos de respuesta de base de datos, promedio de 15 ms [2].
Agilidad en despliegues	Las actualizaciones se realizan de forma independiente por servicio, sin afectar al resto.	De 3 despliegues/semana a 350 despliegues diarios con tiempo de actualización de 5.8 min [2].
Autonomía de equipos	Equipos pequeños gestionan servicios específicos, acelerando la innovación.	Promedio de 3.5 dependencias directas por servicio; reducción del 65 % en acoplamiento [2].

5. Desafíos encontrados

A pesar de los amplios beneficios obtenidos, la transición de Netflix hacia microservicios también enfrentó desafíos significativos que deben considerarse al evaluar la adopción de este tipo de arquitectura. Ignorar estas dificultades sería un análisis incompleto del caso.

Cuadro 2: Desafíos enfrentados y soluciones implementadas por Netflix

Desafío	Descripción	Solución implementada por Netflix
Complejidad en la gestión	A medida que el número de microservicios creció, la orquestación y el monitoreo se volvieron más complejos, requiriendo herramientas especializadas.	Adopción de Netflix Conductor para orquestación de flujos de trabajo y Atlas para monitoreo avanzado [1].
Latencia entre servicios	La dependencia en llamadas API entre servicios introdujo latencia y nuevos puntos de falla en el sistema.	Uso de caché, balanceo de carga y patrones como circuit breakers para mitigar la latencia [1].
Observabilidad distribuida	Rastrear el comportamiento del sistema a través de cientos de servicios independientes es considerablemente más complejo que en un monolito.	Infraestructura de monitoreo que procesa aproximadamente 1.8 billones de métricas diarias, con una tasa de detección de anomalías del 96.5 % [2].
Consistencia de datos	La transición del sistema de datos centralizado a almacenes distribuidos por servicio presentó riesgos de inconsistencia e interrupciones.	Estrategias de migración gradual y uso de bases de datos especializadas por servicio [1].
Fallos en producción	La distribución del sistema amplificó el riesgo de fallos en cascada que podían afectar múltiples servicios simultáneamente.	Implementación de Chaos Engineering con Chaos Monkey para pruebas proactivas de tolerancia a fallos [1].

6. Aplicación al PFC

Los principios observados en la arquitectura de microservicios de Netflix pueden aplicarse al proyecto integrador TiendaTech mediante la división del sistema en servicios independientes responsables de funcionalidades específicas, como gestión de usuarios, catálogo de productos, inventario, pedidos, pagos y notificaciones. Este enfoque permitiría escalar de forma individual los módulos con mayor carga de trabajo, facilitar el mantenimiento del sistema y reducir el impacto de posibles fallos. Además, favorecería la evolución futura de la plataforma al permitir que cada servicio sea desarrollado, actualizado y desplegado de manera independiente.

Referencias

- [1] C. Henríquez, J. D. R. Valencia y G. Sánchez-Torres, “Architectural Evolution from Monolithic to Microservices in Scalable Systems: A Case Study of Netflix,” *Prospec-tiva*, vol. 23, n.º 1, 2025.

- [2] R. Katkuri, “Journey to Scalability and Efficiency with Microservices and Serverless Computing,” *European Journal of Computer Science and Information Technology*, vol. 13, n.º 5, págs. 109-118, abr. de 2025. DOI: [10.37745/ejcsit.2013/vol13n5109118](https://doi.org/10.37745/ejcsit.2013/vol13n5109118).